

Neural Stochastic Differential Equations for Generative Stress-Testing of Cointegrated Pairs Trading Strategies

Killian Guillaume
Independent Research
killian.guillaume@email.com

April 18, 2026

Abstract

We present a generative framework for stress-testing statistical arbitrage strategies on cointegrated asset pairs. The core model is a *Neural Stochastic Differential Equation* (Neural SDE) that learns the joint dynamics of two log-price processes conditioned on a spread deviation signal encoding mean-reversion. The model is trained via a *Kernel Maximum Mean Discrepancy* (MMD) loss computed on truncated path signatures, ensuring distributional convergence in the space of continuous paths rather than pointwise matching. At inference, the trained SDE generates millions of synthetic market trajectories that preserve the cointegration structure, volatility clustering, and tail behavior of real data. These synthetic paths are filtered via the Augmented Dickey–Fuller test and fed into a Monte Carlo backtesting engine that computes Value-at-Risk and Conditional Value-at-Risk under realistic transaction cost models. We describe the full mathematical framework and the architectural design decisions that make this pipeline production-ready.

Keywords: Neural SDE, Generative Model, Pairs Trading, Cointegration, Path Signatures, MMD, Stress Testing, Monte Carlo Simulation

1 Introduction

Pairs trading is a market-neutral strategy that exploits temporary deviations between two cointegrated assets [Vidyamurthy, 2004]. A critical challenge is *stress-testing*: evaluating whether a calibrated strategy is robust to unseen market conditions, liquidity crises, or regime changes. Historical backtesting is limited to a single realized path of the market, which provides no distributional information about strategy performance.

We address this limitation by building a *Neural SDE* [Kidger et al., 2021] that learns the joint stochastic dynamics of two cointegrated price processes. Once trained, the model generates arbitrarily many synthetic market trajectories, each preserving the cointegration relationship, while exhibiting natural variation in volatility, drift, and tail events.

Our key contributions are:

1. A 2-dimensional Neural SDE architecture with a *spread deviation conditioning* mechanism that explicitly encodes mean-reversion inductive bias.
2. Training via *Kernel Signature MMD*, a distributional loss that matches the law of generated paths to real data in the universal feature space of path signatures.
3. A production pipeline that combines the generative model with ADF-based quality filtering and Monte Carlo risk evaluation (VaR, CVaR) under 4 realistic cost scenarios.

2 Problem Formulation

Let P_t^A and P_t^B denote the prices of two assets at time t . We work in log-price space, defining the state vector:

$$X_t = \begin{pmatrix} \log P_t^A \\ \log P_t^B \end{pmatrix} \in \mathbb{R}^2. \quad (1)$$

The *log-spread* is defined as:

$$s_t = \log P_t^A - \log P_t^B. \quad (2)$$

For a cointegrated pair, s_t is a stationary process. The trading signal is derived from the z-score of s_t , and the strategy enters when $|z_t| > \tau_{\text{entry}}$ and exits when $|z_t| < \tau_{\text{exit}}$.

Remark 2.1. *Working in log-price space (rather than raw prices) ensures scale invariance: the same model can generate paths for assets with vastly different nominal prices, since the SDE operates on dimensionless standardized log-prices.*

2.1 Data Preprocessing

Given raw price series $(P_t^A, P_t^B)_{t=1}^T$, we compute:

$$\tilde{X}_t = \frac{X_t - \hat{\mu}}{\hat{\sigma}}, \quad \hat{\mu} = \frac{1}{T} \sum_{t=1}^T X_t, \quad \hat{\sigma} = \sqrt{\frac{1}{T} \sum_{t=1}^T (X_t - \hat{\mu})^2}, \quad (3)$$

where $\hat{\mu}, \hat{\sigma} \in \mathbb{R}^2$ are computed component-wise on the training set only. The validation set uses the training set's statistics to prevent information leakage.

2.2 Strided Sampling

Training sequences are extracted as sliding windows of length L from the standardized time series. With a naïve stride of 1, consecutive windows $\tilde{X}_{t:t+L}$ and $\tilde{X}_{t+1:t+1+L}$ overlap by $L - 1$ points, creating near-duplicate training samples that encourage memorization of micro-transitions rather than learning the underlying generative dynamics.

To enforce statistical independence between training samples, we subsample windows with a stride $\Delta > 1$:

$$\mathcal{D}_{\text{train}} = \left\{ \tilde{X}_{k\Delta:k\Delta+L} \right\}_{k=0}^{\lfloor (T-L)/\Delta \rfloor}. \quad (4)$$

This reduces the dataset size by a factor of Δ but dramatically improves generalization: the model is forced to learn the *distributional law* of the process (cointegration, volatility clustering) rather than memorizing specific sequences. In practice, $\Delta = 8$ provides a good trade-off between sample diversity and dataset size for hourly data.

3 Neural SDE Architecture

3.1 State-Space Formulation

We model the joint dynamics of the standardized log-prices as an Itô SDE:

$$d\tilde{X}_t = f_\theta(\tilde{X}_t, \Delta s_t) dt + g_\theta(\tilde{X}_t, \Delta s_t) dW_t, \quad (5)$$

where:

- $\tilde{X}_t \in \mathbb{R}^2$ is the SDE state (standardized log-prices of assets A and B),
- $f_\theta : \mathbb{R}^2 \rightarrow \mathbb{R}^2$ is the *drift network* (parametric trend),
- $g_\theta : \mathbb{R}^2 \rightarrow \mathbb{R}_{>0}^2$ is the *diffusion network* (parametric local volatility),
- $W_t \in \mathbb{R}^2$ is a standard 2-dimensional Brownian motion (diagonal noise),
- Δs_t is the *spread deviation* signal defined below.

3.2 Spread Deviation Conditioning

Rather than feeding the raw spread $s_t = \tilde{X}_t^{(1)} - \tilde{X}_t^{(2)}$ to the networks, we use the *deviation from the initial spread*:

$$\Delta s_t = s_t - s_0 = \left(\tilde{X}_t^{(1)} - \tilde{X}_t^{(2)} \right) - \left(\tilde{X}_0^{(1)} - \tilde{X}_0^{(2)} \right). \quad (6)$$

This design choice introduces mean-reversion inductive bias: $\Delta s_0 = 0$ at integration start, and the drift network observes only the *tension* (how far the spread has moved from its starting point), not its absolute level. This encourages the drift to learn a restoring force that pulls Δs_t back toward zero.

The extended input to both networks is:

$$\tilde{X}_t^{\text{ext}} = \left(\tilde{X}_t^{(1)}, \tilde{X}_t^{(2)}, \Delta s_t \right) \in \mathbb{R}^3. \quad (7)$$

3.3 Network Architectures

Both f_θ and g_θ are implemented as 3-layer MLPs with SiLU activations:

$$f_\theta : \mathbb{R}^3 \xrightarrow{\text{Linear}(3,H)} \xrightarrow{\text{SiLU}} \xrightarrow{\text{Linear}(H,H)} \xrightarrow{\text{SiLU}} \xrightarrow{\text{Linear}(H,2)} \mathbb{R}^2, \quad (8)$$

$$g_\theta : \mathbb{R}^3 \xrightarrow{\dots(\text{same architecture})} \xrightarrow{\text{softplus}} \mathbb{R}_{>0}^2, \quad (9)$$

where H is the hidden dimension (64 in our experiments). We rely on strided sampling (Section 2) and weight decay rather than dropout for regularization, since dropout introduces stochasticity in the drift and diffusion functions that can destabilize the SDE solver during integration.

Weight initialization. To ensure stability at the start of training:

- f_θ : last layer weights and biases initialized to zero (zero drift at $t = 0$).
- g_θ : last layer bias initialized to $\text{softplus}^{-1}(\sigma_{\text{target}})$ to match a target volatility level.

Positivity of diffusion. The softplus activation $\text{softplus}(x) = \log(1 + e^x)$ guarantees $g_\theta > 0$. An additional clamp $g_\theta \in [\sigma_{\min}, 5.0]$ prevents both numerical underflow and explosion during integration.

3.4 Numerical Integration

The SDE (5) is solved via the Milstein scheme [Kloeden and Platen, 1992] implemented in `torchsde`:

$$\begin{aligned} \tilde{X}_{t+\delta} = & \tilde{X}_t + f_\theta(\tilde{X}_t^{\text{ext}}) \delta + g_\theta(\tilde{X}_t^{\text{ext}}) \Delta W_t \\ & + \frac{1}{2} g_\theta(\tilde{X}_t^{\text{ext}}) g'_\theta(\tilde{X}_t^{\text{ext}}) ((\Delta W_t)^2 - \delta), \end{aligned} \quad (10)$$

where δ is the time step and $\Delta W_t \sim \mathcal{N}(0, \delta I_2)$. The Milstein correction provides strong order 1.0 convergence, compared to 0.5 for Euler–Maruyama.

4 Training via Kernel Signature MMD

4.1 Path Signatures

The *signature* of a path $\mathbf{x} : [0, T] \rightarrow \mathbb{R}^d$ is the sequence of iterated integrals that uniquely characterizes the path up to tree-like equivalence [Lyons, 1998]:

Definition 4.1 (Truncated Path Signature). *For a continuous path $\mathbf{x} \in C([0, T], \mathbb{R}^d)$, the signature truncated at depth N is:*

$$\text{Sig}^{(\leq N)}(\mathbf{x}) = \left(1, \int_0^T d\mathbf{x}_t, \int_0^T \int_0^t d\mathbf{x}_s \otimes d\mathbf{x}_t, \dots \right) \in T^{(\leq N)}(\mathbb{R}^d), \quad (11)$$

where $T^{(\leq N)}(\mathbb{R}^d) = \bigoplus_{k=0}^N (\mathbb{R}^d)^{\otimes k}$ is the truncated tensor algebra.

The signature is a *universal nonlinear feature map* for sequential data: any continuous function of the path can be approximated as a linear function of its signature [Király and Oberhauser, 2019]. We compute truncated signatures at depth $N = 3$ using the **signatory** library, yielding a feature vector of dimension $\sum_{k=1}^3 d^k = 2 + 4 + 8 = 14$ for $d = 2$.

4.2 Kernel MMD Loss

Definition 4.2 (Maximum Mean Discrepancy). *Given two distributions $\mathbb{P}$ and $\mathbb{Q}$ and a reproducing kernel k , the squared MMD is:*

$$\text{MMD}_k^2(\mathbb{P}, \mathbb{Q}) = \mathbb{E}_{X, X' \sim \mathbb{P}}[k(X, X')] + \mathbb{E}_{Y, Y' \sim \mathbb{Q}}[k(Y, Y')] - 2 \mathbb{E}_{X \sim \mathbb{P}, Y \sim \mathbb{Q}}[k(X, Y)]. \quad (12)$$

We apply the MMD on the space of path signatures. Given a batch of B real paths $\{\mathbf{x}^{(i)}\}_{i=1}^B$ and B generated paths $\{\hat{\mathbf{x}}^{(i)}\}_{i=1}^B$:

1. Compute truncated signatures: $\phi_i = \text{Sig}^{(\leq N)}(\mathbf{x}^{(i)})$, $\hat{\phi}_i = \text{Sig}^{(\leq N)}(\hat{\mathbf{x}}^{(i)})$.
2. Normalize to the unit sphere: $\bar{\phi}_i = \phi_i / \|\phi_i\|_2$.
3. Compute the empirical MMD with a Gaussian (RBF) kernel:

$$k(\bar{\phi}, \bar{\psi}) = \exp\left(-\frac{\|\bar{\phi} - \bar{\psi}\|^2}{2\sigma^2}\right) = \exp\left(-\frac{2 - 2\langle \bar{\phi}, \bar{\psi} \rangle}{2\sigma^2}\right), \quad (13)$$

where the simplification uses $\|\bar{\phi}\| = \|\bar{\psi}\| = 1$.

The loss function is:

$$\mathcal{L}_{\text{Sig-MMD}} = \frac{1}{B^2} \sum_{i,j} k(\bar{\phi}_i, \bar{\phi}_j) + \frac{1}{B^2} \sum_{i,j} k(\hat{\phi}_i, \hat{\phi}_j) - \frac{2}{B^2} \sum_{i,j} k(\bar{\phi}_i, \hat{\phi}_j). \quad (14)$$

Proposition 4.3. $\mathcal{L}_{\text{Sig-MMD}} = 0$ if and only if the distribution of generated paths matches the distribution of real paths in the signature feature space. Since the signature is a universal feature for paths, this implies convergence of all continuous path functionals.

4.3 Training Procedure

Algorithm 1 Training the Neural SDE with Kernel Signature MMD

Require: Training data $\{(\tilde{X}_{0:L}^{(i)})\}_{i=1}^N$, learning rate η , signature depth N_s

```

1: for epoch = 1, ...,  $E_{\max}$  do
2:   for each batch  $\{\tilde{X}_{0:L}^{(b)}\}_{b=1}^B$  do
3:      $y_0^{(b)} \leftarrow \tilde{X}_0^{(b)}$  ▷ Initial state from real data
4:      $\hat{X}_{0:L}^{(b)} \leftarrow \text{sdeint}(f_\theta, g_\theta, y_0^{(b)}, t_{0:L})$  ▷ Milstein integration
5:      $\bar{\phi}^{(b)} \leftarrow \text{Normalize}\left(\text{Sig}^{(\leq N_s)}(\tilde{X}^{(b)})\right)$ 
6:      $\hat{\phi}^{(b)} \leftarrow \text{Normalize}\left(\text{Sig}^{(\leq N_s)}(\hat{X}^{(b)})\right)$ 
7:      $\mathcal{L} \leftarrow \text{MMD}_{\text{RBF}}^2(\bar{\phi}, \hat{\phi})$ 
8:      $\theta \leftarrow \theta - \eta \cdot \text{clip}(\nabla_\theta \mathcal{L}, 1.0)$ 
9:   end for
10:  if epoch mod 10 = 0 then
11:    Compute validation loss; update scheduler; check early stopping
12:  end if
13: end for

```

5 Inference and Quality Control

5.1 Synthetic Path Generation

At inference, given the last observed standardized log-prices $\tilde{X}_T$:

$$y_0 = \tilde{X}_T = \frac{X_T - \hat{\mu}}{\hat{\sigma}}, \quad (15)$$

we integrate the SDE forward to produce K synthetic paths of length L :

$$\{\hat{X}_{T:T+L}^{(k)}\}_{k=1}^K, \quad K \gg 1. \quad (16)$$

Each generated path is de-standardized:

$$\hat{P}_t^A = \exp\left(\hat{\sigma}_A \cdot \hat{X}_t^{(1)} + \hat{\mu}_A\right), \quad \hat{P}_t^B = \exp\left(\hat{\sigma}_B \cdot \hat{X}_t^{(2)} + \hat{\mu}_B\right). \quad (17)$$

5.2 ADF-Based Quality Filtering

Not all generated paths preserve cointegration over long horizons. We filter using the Augmented Dickey–Fuller (ADF) test:

Definition 5.1 (Acceptance criterion). *A synthetic path k is accepted if and only if:*

$$\text{ADF-pvalue}\left(\hat{s}_{T:T+L}^{(k)}\right) < \alpha, \quad \alpha = 0.05, \quad (18)$$

where $\hat{s}_t^{(k)} = \log \hat{P}_t^{A,(k)} - \log \hat{P}_t^{B,(k)}$ is the log-spread of the generated path.

The acceptance rate $r = |\{k : \text{accepted}\}|/K$ serves as a diagnostic: $r \approx 1$ indicates the SDE has fully learned the cointegration dynamics; $r \ll 1$ indicates the model is diverging on long simulations.

6 Monte Carlo Stress-Testing

6.1 Strategy Evaluation Framework

For each accepted synthetic path k :

1. Compute the Kalman-filtered spread $\hat{s}_t^{(k)}$ and its z-score $z_t^{(k)}$.
2. Generate entry/exit signals using threshold crossings.
3. Execute the backtest under a cost model parameterized by a fee scenario $\mathcal{S}$.
4. Record performance metrics: Sharpe ratio, maximum drawdown, Calmar ratio, profit factor.

6.2 Realistic Transaction Cost Model

Each round-trip trade on a pair involves 4 transactions (open A, open B, close A, close B). The total cost per round-trip is:

$$C_{\text{RT}} = 4 \times N_{\text{leg}} \times (\phi_{\text{fee}} + \phi_{\text{slip}}), \quad (19)$$

where N_{leg} is the notional per leg, ϕ_{fee} is the taker fee rate, and ϕ_{slip} is the slippage in basis points. For positions held over multiple bars, a funding rate ρ_{8h} is applied:

$$C_{\text{funding}}(t) = 2 \times N_{\text{leg}} \times |\text{pos}_t| \times \rho_{8h} \times \frac{\Delta t}{8h}. \quad (20)$$

We evaluate under 4 scenarios ranging from optimistic (2 bps maker, VIP tier) to stressed (10 bps taker + 15 bps slippage + extreme funding).

6.3 Risk Metrics

Given the distribution of Sharpe ratios $\{\mathcal{R}^{(k)}\}_{k=1}^K$ across all accepted paths:

$$\text{VaR}_{95\%} = \inf\{x : \mathbb{P}(\mathcal{R} \leq x) \geq 0.05\}, \quad (21)$$

$$\text{CVaR}_{95\%} = \mathbb{E}[\mathcal{R} \mid \mathcal{R} \leq \text{VaR}_{95\%}]. \quad (22)$$

The CVaR (Expected Shortfall) provides a coherent risk measure that captures the expected performance in the worst 5% of generated scenarios — a quantity inaccessible from a single historical backtest.

7 Architecture Overview

8 Implementation Details

The model is trained on AWS EC2 Spot instances (g4dn.xlarge) with CUDA acceleration. Training telemetry is logged via Weights & Biases. The codebase uses `torchsde` for SDE integration and `signatory` for GPU-accelerated signature computation.

9 Discussion

Why Neural SDE over autoregressive models? Autoregressive generative models (e.g., GPT-style, or our previous Conditional Flow Matching approach) generate paths as sequences of discrete steps, requiring *stitching* for long-horizon generation. The SDE formulation integrates continuously, producing paths of arbitrary length in a single forward solve without accumulating stitching artifacts.

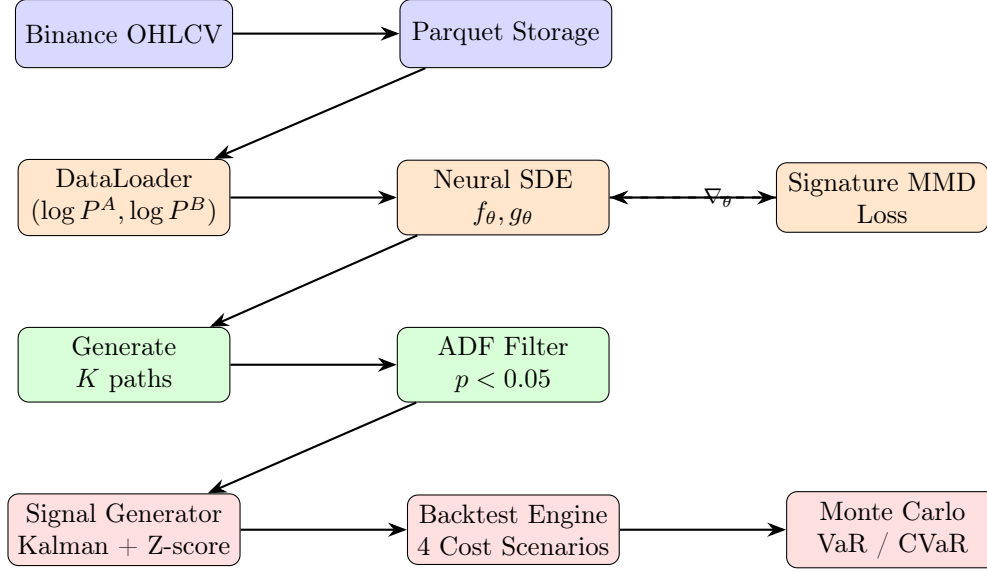


Figure 1: End-to-end pipeline: from raw data ingestion to Monte Carlo risk evaluation.

Why distributional matching via signatures? Pointwise losses (MSE on returns, log-likelihood) match individual trajectories but do not guarantee that the *distribution* of generated paths matches reality. The Kernel Signature MMD operates on the space of path measures, ensuring that all continuous functionals of the path — including volatility clustering, autocorrelation structure, and tail behavior — are replicated.

Role of spread deviation conditioning. Without Δs_t , the drift and diffusion networks would need to implicitly learn the cointegration relationship from the two log-prices alone. By explicitly providing the spread deviation as an auxiliary input, we inject domain knowledge about mean-reversion into the architecture, significantly reducing the sample complexity of training.

10 Conclusion

We have presented a Neural SDE framework for generating synthetic cointegrated market paths to stress-test pairs trading strategies. The key innovations are:

1. *Spread deviation conditioning*: encoding mean-reversion directly in the SDE architecture.
2. *Kernel Signature MMD training*: distributional matching in the universal feature space of path signatures.
3. *ADF quality filtering*: post-generation stationarity enforcement for robust Monte Carlo evaluation.

This framework transforms the model evaluation paradigm from “how did the strategy perform on one historical path?” to “what is the distribution of strategy performance across thousands of plausible market scenarios?”, enabling statistically rigorous risk assessment.

References

- P. Kidger, J. Foster, X. Li, and T. Lyons. Neural SDEs as infinite-dimensional GANs. In *Proceedings of the 38th International Conference on Machine Learning (ICML)*, 2021.
- T. Lyons. Differential equations driven by rough paths. *Revista Matemática Iberoamericana*, 14(2):215–310, 1998.

Table 1: Hyperparameters used in experiments.

Component	Parameter	Value
Neural SDE	State dimension d	2
	Hidden dimension H	64
	Target volatility σ_{target}	1.0
Training	Sequence length L	256
	Stride Δ	8
	Batch size B	128
	Signature depth N_s	3
	Learning rate	10^{-3} (AdamW)
	Weight decay	10^{-2}
	Early stopping patience	70 epochs
	Integration method	Milstein
Data	Asset pair	ADA/USDT $\times$ AVAX/USDT
	Timeframe	1 hour
	Train/Val split	80% / 20% (chronological)

F. Király and H. Oberhauser. Kernels for sequentially ordered data. *Journal of Machine Learning Research*, 20(31):1–45, 2019.

P. E. Kloeden and E. Platen. *Numerical Solution of Stochastic Differential Equations*. Springer-Verlag, 1992.

G. Vidyamurthy. *Pairs Trading: Quantitative Methods and Analysis*. Wiley, 2004.

I. Chevyrev and H. Oberhauser. Signature moments to characterize laws of stochastic processes. *Journal of Machine Learning Research*, 23(176):1–42, 2022.

A. Gretton, K. M. Borgwardt, M. J. Rasch, B. Schölkopf, and A. Smola. A kernel two-sample test. *Journal of Machine Learning Research*, 13:723–773, 2012.